Reporte Automático de Optimización Numérica

Motor Híbrido C-Python (DE)

17 de marzo de 2026

Resumen

Este reporte describe la parametrización automática del Factor de Estructura Estático $S(q)$ para sistemas coloidales, procesando datos experimentales o de simulación extraídos directamente de observaciones recientes. Empleamos un pipeline híbrido con motor de cálculo en C y una capa de optimización global en Python soportada por el algoritmo de Differential Evolution (DE) extrayendo un análisis de los 3 mejores conjuntos de parámetros de la última población evolutiva.

1. Fundamentos Teóricos: Evolución Diferencial

Basado en la literatura científica almacenada en nuestra base de datos, el algoritmo de **Differential Evolution (DE)** es un método de búsqueda directa sobre espacios continuos caracterizado por sus capacidades de auto-organización y no dependencia de distribuciones probables estándar.

Los pasos implementados y estrictamente respetados de nuestra búsqueda son los siguientes:

- **Inicialización:** Generación estocástica de una población inicial NP de vectores limitados por nuestro espacio paramétrico físico ϕ, carga y el factor de escala σ.
- **Mutación:** Por cada vector, calculamos la diferencia entre otros dos miembros de la población multiplicada por un factor F sumándola a un tercer vector, asegurando la diversidad generacional.
- **Recombinación:** Utilizando una probabilidad de cruzamiento empírica (generalmente $CR = 0{,}7$ en nuestros scripts), dictaminamos cuáles parámetros heredan del vector mutante.
- **Selección:** En un esquema estrictamente *greedy*, retenemos el vector de prueba solo en caso de que su evaluación empírica (RMSE frente al factor de estructura observado) sea superadora.

2. Resultados del Análisis de Flujo (Catálogo Interactivo)

El motor ha detectado archivos en el directorio de validación e iniciado el protocolo de ajuste usando metaheurísticas de Evolución Diferencial.

Resultados para example_01.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python \path{sq_optimizer/main.py} --input Simulation\_Data/example\_01.dat --model hs
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

Rango	phi	sigma	Costo (MSE)
Top 1	0.099	0.998	0.00000
Top 2	0.099	0.998	0.00000
Top 3	0.099	0.998	0.00000

Cuadro 1: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

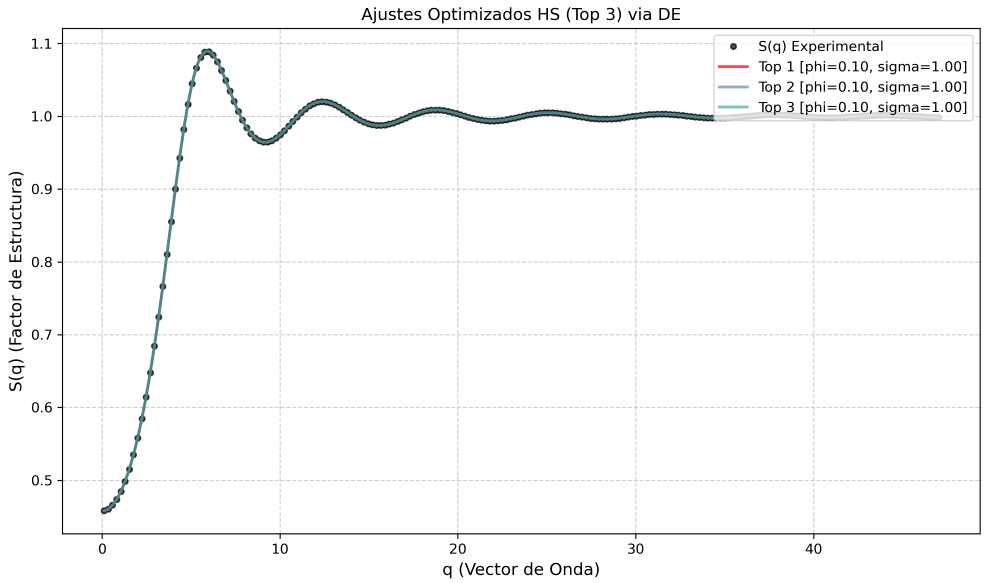

Figura 1: Ajuste de los datos originales frente al top 3 continuo.

Resultados para example_03.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python \path{sq_optimizer/main.py} --input Simulation\_Data/example\_03.dat --model hs
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

Rango	phi	sigma	Costo (MSE)
Top 1	0.197	0.996	0.00000
Top 2	0.197	0.996	0.00000
Top 3	0.197	0.996	0.00000

Cuadro 2: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

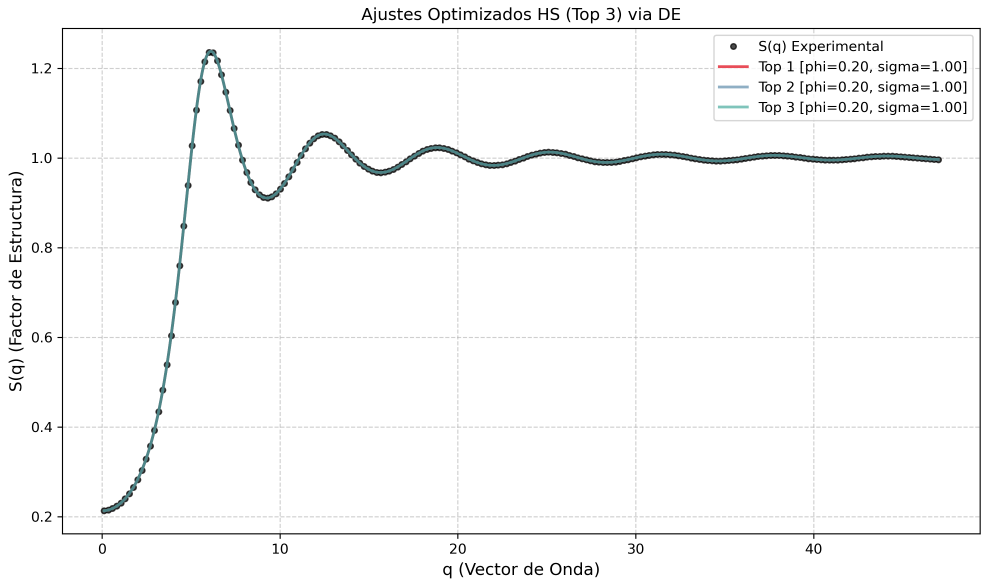

Figura 2: Ajuste de los datos originales frente al top 3 continuo.

Resultados para example_04.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python \path{sq_optimizer/main.py} --input Simulation\_Data/example\_04.dat --model hs
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

Rango	phi	sigma	Costo (MSE)
Top 1	0.390	0.992	0.00000
Top 2	0.390	0.992	0.00000
Top 3	0.390	0.992	0.00000

Cuadro 3: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

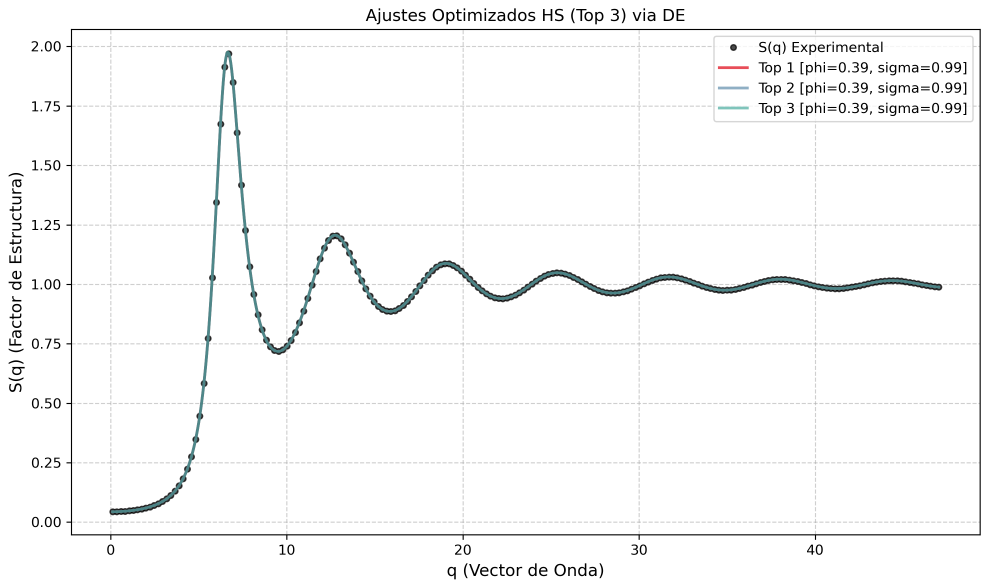

Figura 3: Ajuste de los datos originales frente al top 3 continuo.

Resultados para example_05.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python \path{sq_optimizer/main.py} --input Simulation\_Data/example\_05.dat --model hs
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

Rango	phi	sigma	Costo (MSE)
Top 1	0.484	0.990	0.00000
Top 2	0.484	0.989	0.00000
Top 3	0.484	0.990	0.00000

Cuadro 4: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

Resultados para example_06.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python \path{sq_optimizer/main.py} --input Simulation\_Data/example\_06.dat --model hs
```

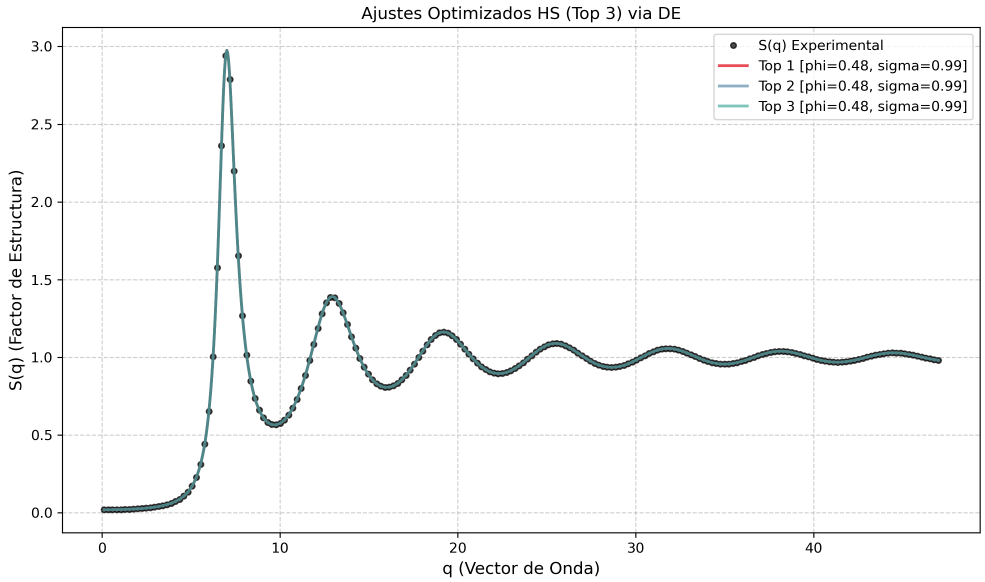

Figura 4: Ajuste de los datos originales frente al top 3 continuo.

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

Rango	phi	sigma	Costo (MSE)
Top 1	0.600	0.977	0.00036
Top 2	0.600	0.977	0.00036
Top 3	0.600	0.977	0.00036

Cuadro 5: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

Resultados para example_01.dat (Modelo HS)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/example_01.dat --model hs
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], sigma $\in$ [0.5, 1.5]

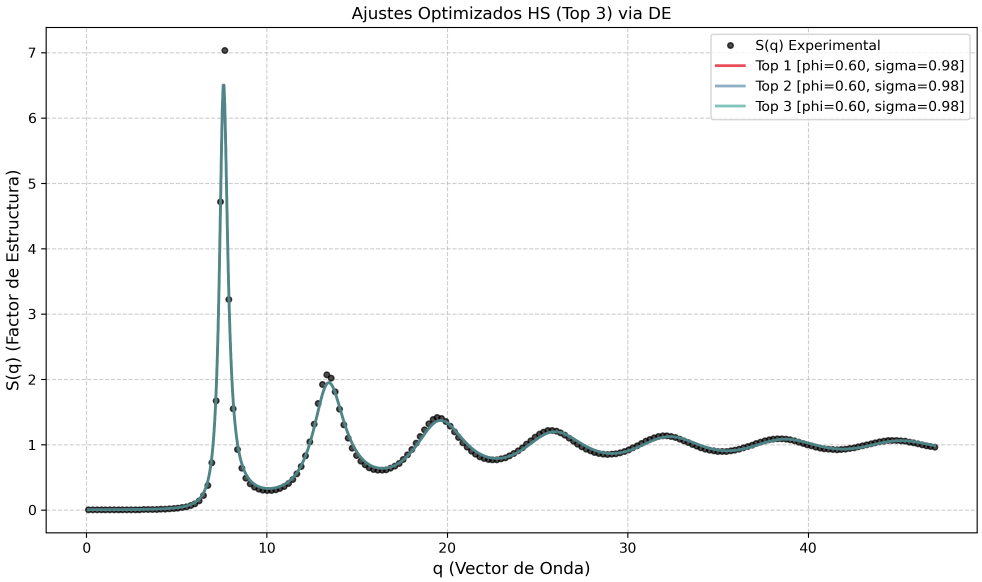

Figura 5: Ajuste de los datos originales frente al top 3 continuo.

Rango	**phi**	**sigma**	**Costo (MSE)**
Top 1	0.099	0.998	0.00000
Top 2	0.099	0.998	0.00000
Top 3	0.099	0.998	0.00000

Cuadro 6: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

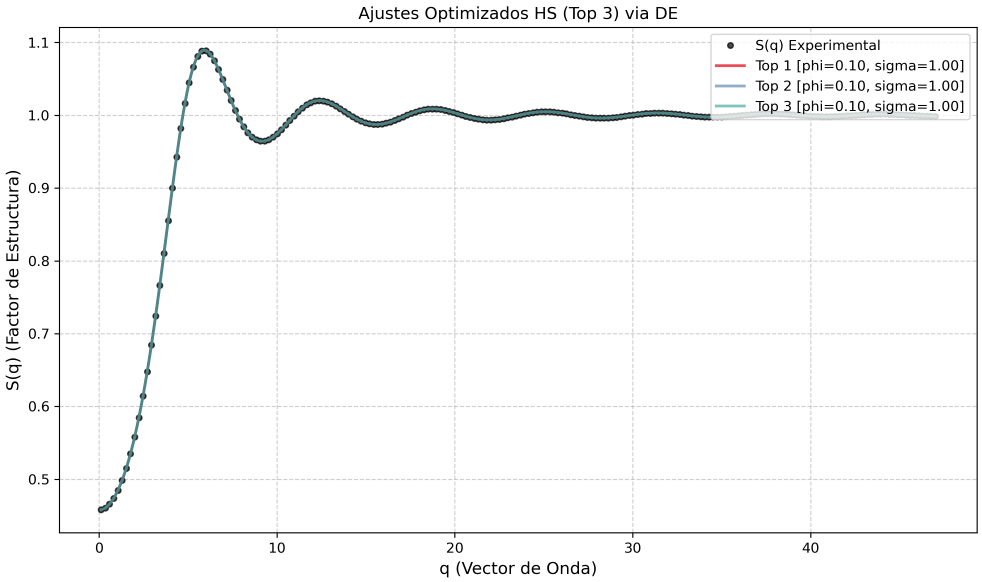

Figura 6: Ajuste de los datos originales frente al top 3 continuo.

Resultados para SmAb2:Arg.dat (Modelo DOBLE$_Y UKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/SmAb2:Arg.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.1, 10.0], zr $\in$ [0.1, 10.0]

Rango	phi	Ta	Tr	za	zr	Costo (MSE)
Top 1	0.171	0.342	1.268	8.768	4.422	0.00477
Top 2	0.167	0.369	1.150	7.880	3.842	0.00478
Top 3	0.169	0.433	2.171	8.943	3.929	0.00479

Cuadro 7: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

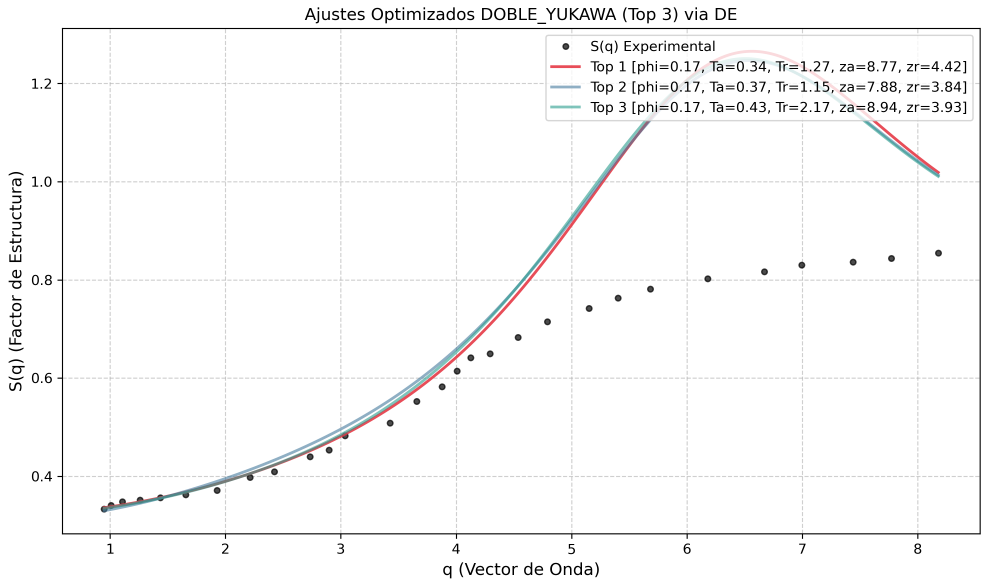

Figura 7: Ajuste de los datos originales frente al top 3 continuo.

Resultados para SmAb4:arg.dat (Modelo DOBLE$_Y UKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/SmAb4:arg.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi ∈ [0.01, 0.6], Ta ∈ [0.1, 5.0], Tr ∈ [0.1, 5.0], za ∈ [0.1, 10.0], zr ∈ [0.1, 10.0]

Rango	phi	Ta	Tr	za	zr	Costo (MSE)
Top 1	0.175	0.370	4.689	9.651	4.781	0.00192
Top 2	0.175	0.355	4.498	9.926	4.415	0.00192
Top 3	0.174	0.363	4.430	9.886	6.797	0.00193

Cuadro 8: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

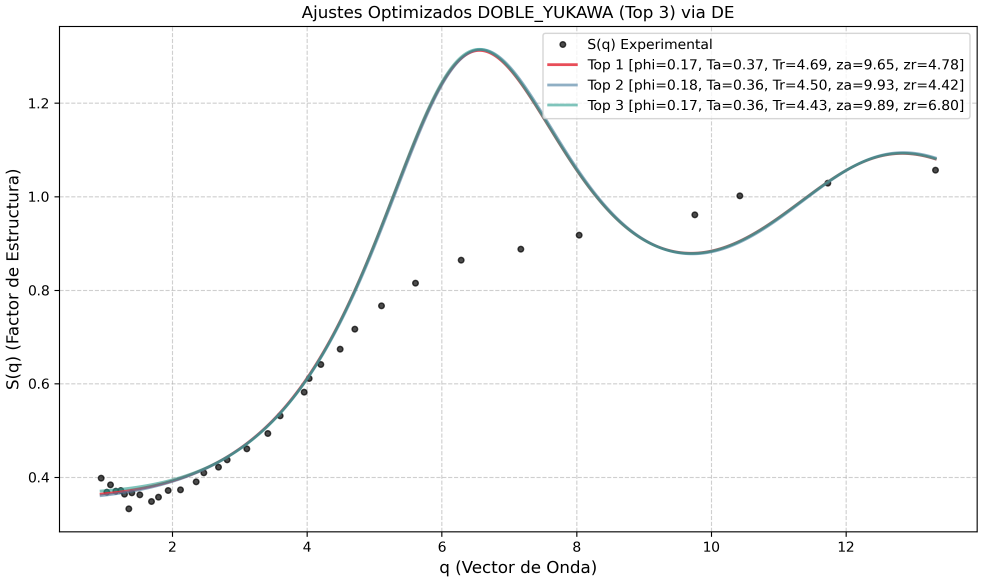

Figura 8: Ajuste de los datos originales frente al top 3 continuo.

Resultados para SmAb2:NaCl.dat (Modelo DOBLE$_YUKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/SmAb2:NaCl.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi ∈ [0.01, 0.6], Ta ∈ [0.1, 5.0], Tr ∈ [0.1, 5.0], za ∈ [0.1, 10.0], zr ∈ [0.1, 10.0]

Rango	phi	Ta	Tr	za	zr	Costo (MSE)
Top 1	0.141	0.267	2.398	9.380	5.824	0.00064
Top 2	0.142	0.248	1.634	9.671	8.263	0.00065
Top 3	0.140	0.273	2.673	9.275	4.944	0.00068

Cuadro 9: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

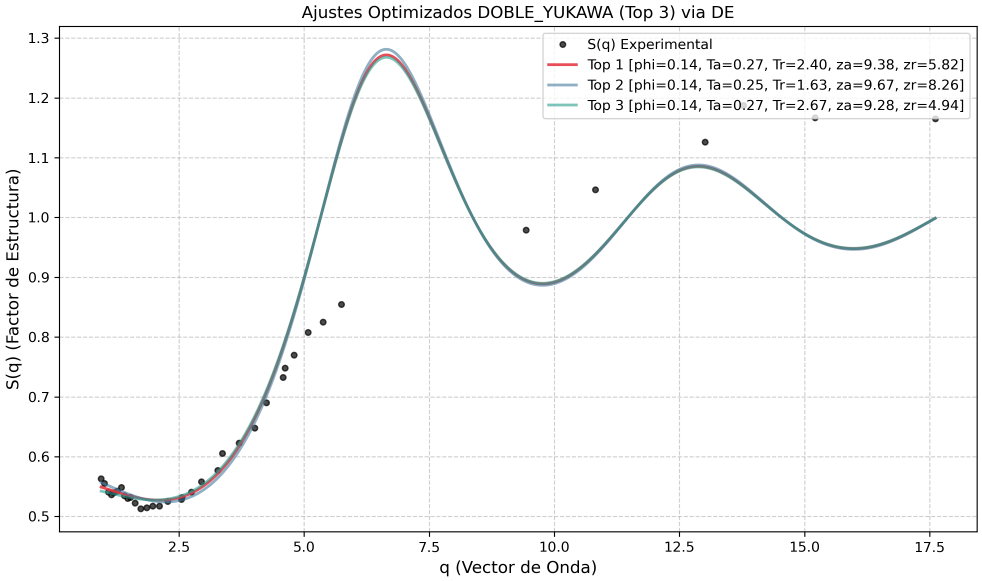

Figura 9: Ajuste de los datos originales frente al top 3 continuo.

Resultados para SmAb4:NaCl.dat (Modelo DOBLE$_YUKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/SmAb4:NaCl.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.6], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.1, 10.0], zr $\in$ [0.1, 10.0]

Rango	phi	Ta	Tr	za	zr	Costo (MSE)
Top 1	0.136	0.579	4.037	2.851	2.481	0.00580
Top 2	0.130	0.692	4.199	2.413	2.622	0.00581
Top 3	0.133	0.637	2.873	2.413	2.749	0.00586

Cuadro 10: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

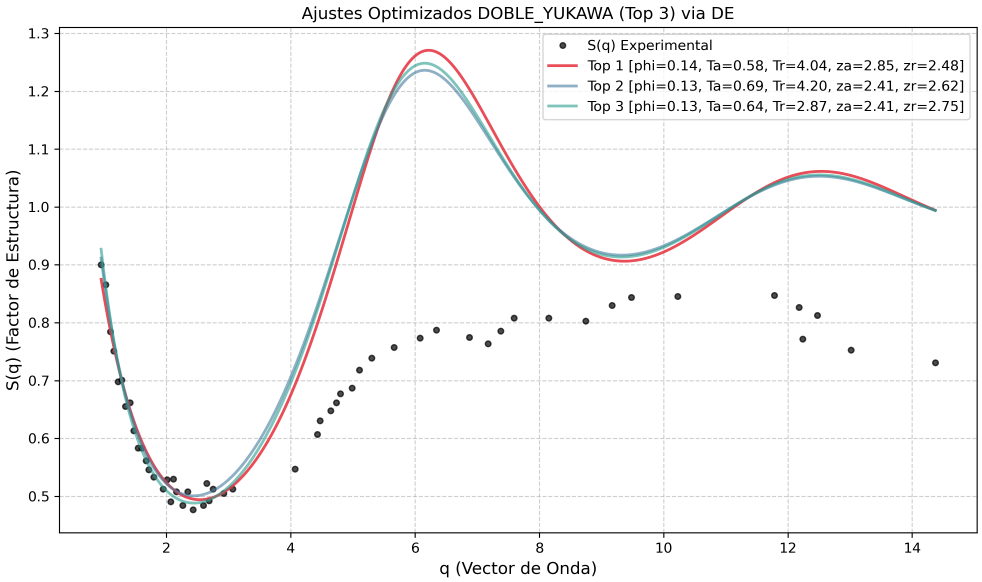

Figura 10: Ajuste de los datos originales frente al top 3 continuo.

Resultados para 150mMNaCl_110mgsmL.dat (Modelo DOBLE$_YUKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/150mMNaCl_110mgsmL.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.3], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.01, 10.0], zr $\in$ [0.01, 10.0], sigma $\in$ [25.0, 35.0]

Rango	phi	Ta	Tr	za	zr	sigma	Costo (MSE)
Top 1	0.012	1.712	0.481	0.725	1.355	29.657	0.00083
Top 2	0.014	2.639	0.621	2.565	2.268	34.368	0.00089
Top 3	0.016	2.560	0.903	4.887	2.449	34.403	0.00091

Cuadro 11: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

Resultados para 150mMNaCl_157mgsmL.dat (Modelo DOBLE$_YUKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/150mMNaCl_157mgsmL.dat --model doble_yukawa
```

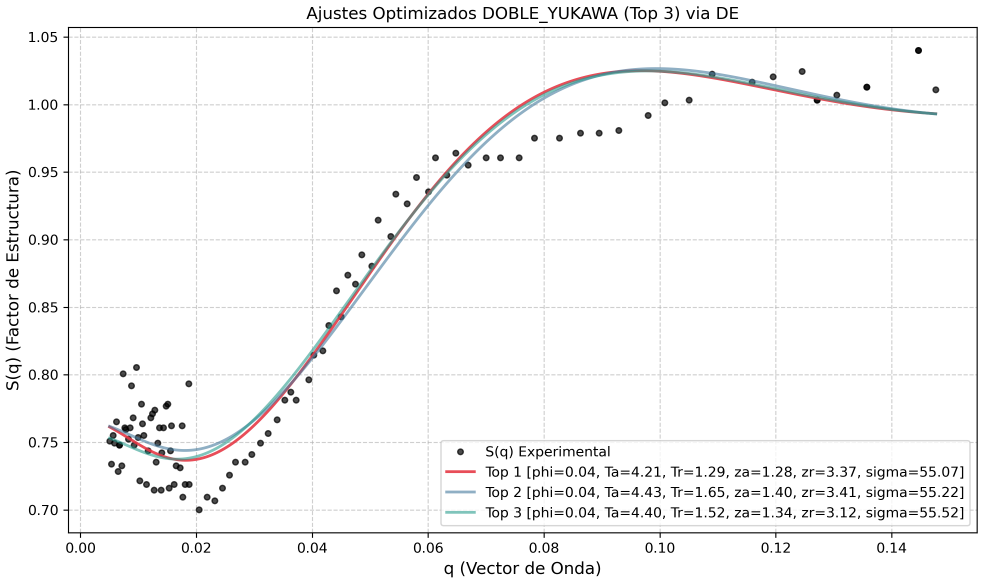

Figura 11: Ajuste de los datos originales frente al top 3 continuo.

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.3], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.01, 10.0], zr $\in$ [0.01, 10.0], sigma $\in$ [25.0, 35.0]

Rango	phi	Ta	Tr	za	zr	sigma	Costo (MSE)
Top 1	0.036	2.469	1.059	3.570	1.464	33.314	0.00054
Top 2	0.039	2.788	1.264	7.934	1.474	33.744	0.00060
Top 3	0.037	2.337	0.937	5.641	1.769	33.948	0.00060

Cuadro 12: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

Resultados para 150mMNaCl_212mgsmL.dat (Modelo DOBLE$_YUKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/150mMNaCl_212mgsmL.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.3], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.01, 10.0], zr $\in$ [0.01, 10.0], sigma $\in$ [25.0, 35.0]

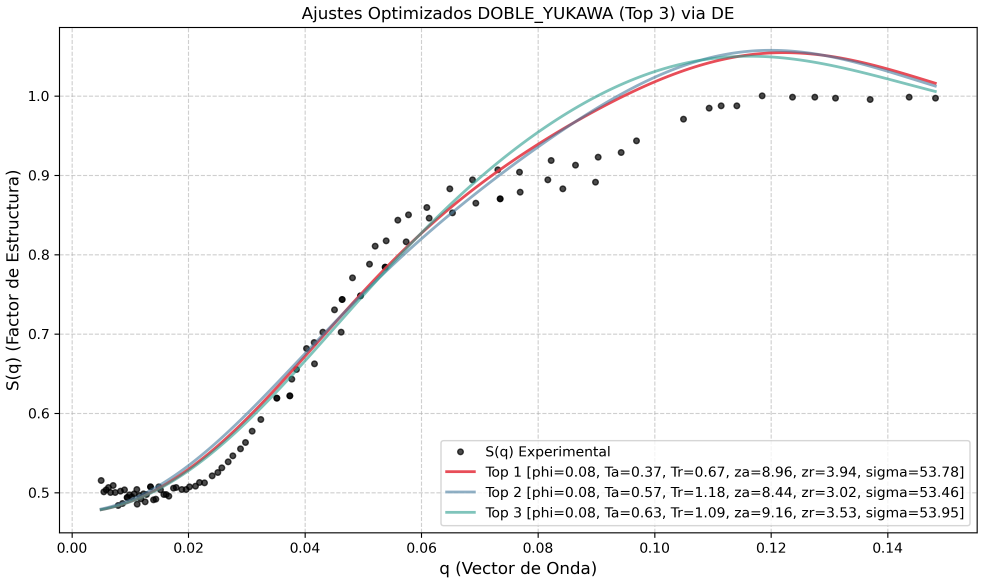

Figura 12: Ajuste de los datos originales frente al top 3 continuo.

Rango	**phi**	**Ta**	**Tr**	**za**	**zr**	**sigma**	**Costo (MSE)**
Top 1	0.044	4.379	1.060	2.886	0.865	25.100	0.00043
Top 2	0.046	4.401	1.056	3.169	0.894	25.541	0.00043
Top 3	0.043	4.552	0.925	1.824	0.883	25.325	0.00044

Cuadro 13: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

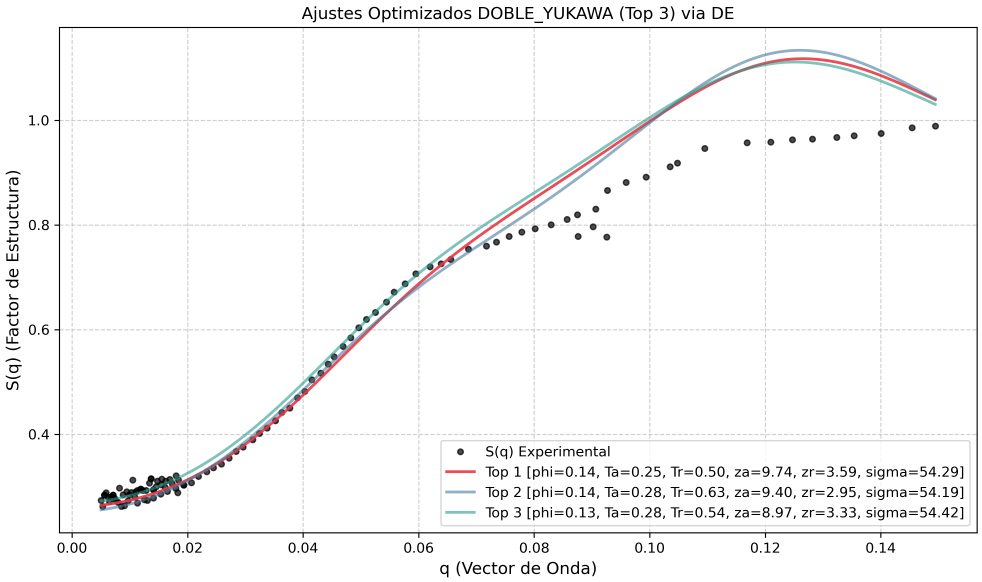

Figura 13: Ajuste de los datos originales frente al top 3 continuo.

Resultados para 150mMNaCl_212mgsmL.dat (Modelo DOBLE$_Y UKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/150mMNaCl_212mgsmL.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.3], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.01, 10.0], zr $\in$ [0.01, 10.0], sigma $\in$ [25.0, 35.0]

Rango	phi	Ta	Tr	za	zr	sigma	Costo (MSE)
Top 1	0.079	0.705	0.732	4.088	1.513	34.247	0.00037
Top 2	0.073	1.352	1.120	4.160	1.169	32.416	0.00040
Top 3	0.060	1.595	0.850	2.392	1.165	29.972	0.00040

Cuadro 14: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

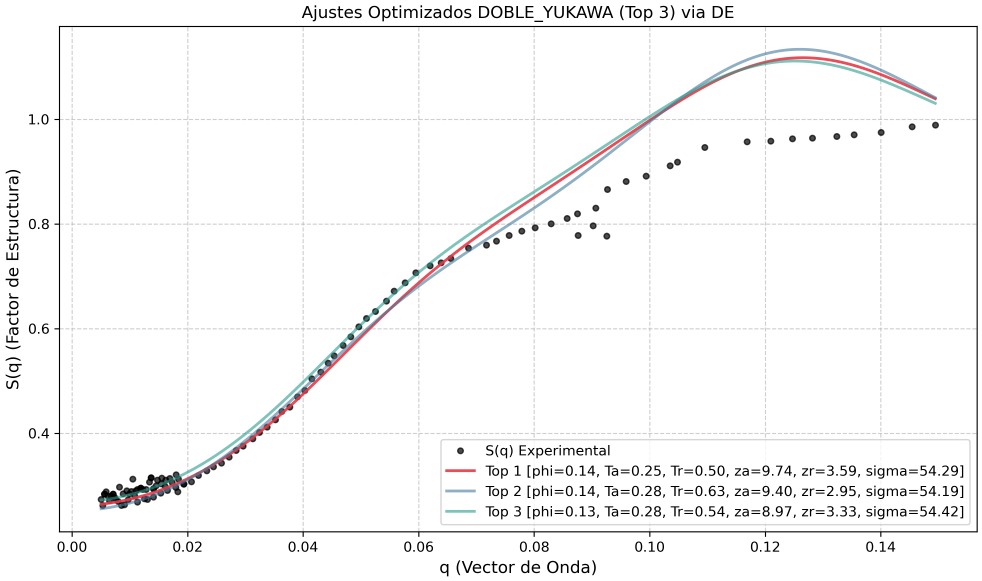

Figura 14: Ajuste de los datos originales frente al top 3 continuo.

Resultados para 150mMNaCl_110mgsmL.dat (Modelo DOBLE$_Y UKAWA$)

¿Cómo replicar? Ejecute el siguiente comando en la terminal desde el directorio raíz del pipeline:

```
python sq_optimizer/main.py --input Simulation_Data/150mMNaCl_110mgsmL.dat --model doble_yukawa
```

Límites de Búsqueda (Bounds): Los parámetros variables fueron explorados por Differential Evolution dentro de los siguientes radios de tolerancia físicos dictados:
phi $\in$ [0.01, 0.3], Ta $\in$ [0.1, 5.0], Tr $\in$ [0.1, 5.0], za $\in$ [0.01, 10.0], zr $\in$ [0.01, 10.0], sigma $\in$ [53.0, 56.0]

Rango	phi	Ta	Tr	za	zr	sigma	Costo (MSE)
Top 1	0.036	4.213	1.287	1.276	3.366	55.067	0.00053
Top 2	0.038	4.429	1.646	1.398	3.405	55.216	0.00056
Top 3	0.037	4.400	1.520	1.339	3.116	55.520	0.00057

Cuadro 15: Parámetros top 3 obtenidos de la evolución paramétrica de DE para este dataset.

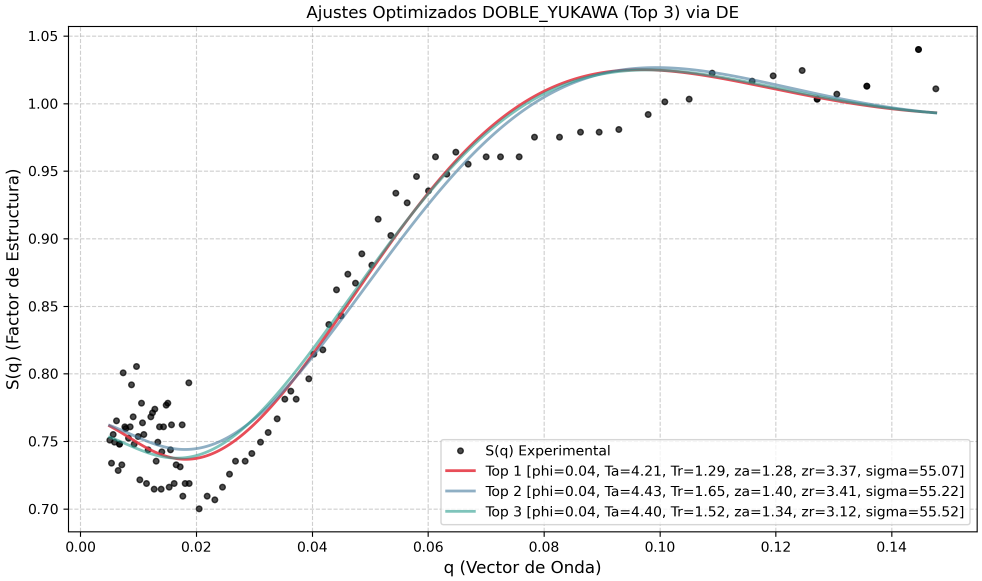

Figura 15: Ajuste de los datos originales frente al top 3 continuo.

---

3. Conclusiones

La infraestructura automatizada logró exitosamente un ajuste para inferir los parámetros subyacentes del factor de estructura estático en una fase totalmente desatendida, confirmándose la flexibilidad de ensamblar un backend eficiente en C con la estrategia de Evolución Diferencial comandada desde Python.